

DEVOPS E ARQUITETURA CLOUD

FASE 2 | AULA 04 -

VOLUMES E REDES

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
MERCADO, CASES E TENDÊNCIAS	15
O QUE VOCÊ VIU NESTA AULA?	16
REFERÊNCIAS.....	17

O QUE VEM POR AÍ?

Você já percebeu que, quando um container é destruído, tudo o que estava dentro dele simplesmente some? Pois é. E também já deve ter notado que, para apps mais complexos, os containers precisam “conversar” entre si — como uma API batendo num banco de dados ou em um serviço de autenticação.

Essa aula é o ponto de virada para quem quer levar containers para o mundo real: você vai aprender a salvar dados de forma segura e com volumes, assim como conectar múltiplos containers de maneira organizada usando redes Docker. A gente vai fazer isso de um jeito simples, com teoria na medida certa e muita prática, mostrando o que realmente importa no dia a dia de quem desenvolve e sobe apps em um ambiente containerizado.

E mais: além de entender os conceitos, você vai praticar criando volumes, montando redes personalizadas e resolvendo problemas reais como um dev experiente.

Se prepara, porque depois dessa aula seus containers vão ficar muito mais robustos, estáveis e profissionais.

HANDS ON

Agora é a hora de ver tudo funcionando de verdade. Nesta aula prática, vamos entender como persistir dados de containers e como eles se comunicam entre si em redes isoladas e organizadas.

Você vai entender na prática por que volumes são essenciais em qualquer app sério — e também como criar redes sob medida para o seu ambiente de containers. Vamos ver o que é possível, o que é recomendável e o que pode causar problema caso você ignore.

Na primeira parte da aula, vamos ver por que containers perdem dados, o que são volumes e como isso tudo funciona na prática. Você vai aprender o básico de como criar volumes, inspecionar e remover — tudo de forma simples, com exemplos reais.

No segundo vídeo, vamos subir containers com volumes montados, usar o -v, explorar o conteúdo desses volumes e depois mergulhar nas redes Docker. Você vai conhecer os tipos de redes (bridge, host e none), entender o papel de cada uma e quando usar.

No terceiro vídeo, é hora de ir além: vamos criar redes personalizadas, conectando containers, testando comunicação entre serviços e discutindo boas práticas. Tudo com foco em um ambiente robusto e escalável, como deve ser em uma produção.

SAIBA MAIS

Volumes Docker: funcionamento interno e drivers

Quando falamos em persistência de dados no Docker, inevitavelmente estamos falando de volumes. Eles são a forma oficial, segura e recomendada de manter dados fora do ciclo de vida efêmero dos containers. Mas o que realmente acontece por trás do `docker volume create`?

Como o Docker gerencia volumes

Quando você cria um volume, o Docker registra esse recurso em seu próprio mecanismo de metadados interno. Ele sabe onde o volume está no sistema de arquivos do host, qual driver está usando e qual container está usando aquele volume. Por padrão, esse volume é armazenado em um caminho como:

<code>/var/lib/docker/volumes/[nome_do_volume]/_data</code>

Comando de Prompt 1 – Volume sendo armazenado no caminho

Fonte: elaborado pelo autor (2025)

Esse `_data` é aquilo que o container enxerga quando você monta o volume, e o Docker se encarrega de conectar esse diretório ao caminho interno do container que você definiu, como por exemplo `/var/lib/mysql`.

Se estiver usando uma engine diferente, como `containerd` ou `cri-o`, o funcionamento geral segue a mesma lógica, mas o caminho físico pode variar.

O que são drivers de volume

O Docker permite usar drivers diferentes para gerenciar onde e como os dados são armazenados. Cada driver implementa uma forma específica de criar, montar e gerenciar volumes. Os mais comuns são:

- **local (padrão):** usa o sistema de arquivos do host. Simples, rápido e suficiente para a maioria dos casos;
- **nfs:** permite montar volumes armazenados em servidores NFS, útil para clusters e compartilhamento em rede;
- **tmpfs:** armazena tudo na RAM e desaparece ao reiniciar. Serve para dados temporários e testes;

- **drivers de terceiros:** como rexray, portworx, que integram com soluções de storage enterprise como EBS, Azure Disk, Ceph, entre outros.

Você escolhe o driver no momento da criação:

```
docker volume create --driver local meu-volume
```

Comando de Prompt 2 – Escolhendo o driver no momento da criação

Fonte: elaborado pelo autor (2025)

Ou, para um volume NFS:

```
docker volume create \
  --driver local \
  --opt type=nfs \
  --opt o=addr=192.168.1.100,rw \
  --opt device=:/dados/nfs \
  volume-nfs
```

Comando de Prompt 3 – Volume NFS

Fonte: elaborado pelo autor (2025)

Como escolher o driver ideal

Depende do seu cenário:

- **Ambiente de desenvolvimento:** use o driver **local**. É simples e resolve bem;
- **Ambiente de produção com múltiplos hosts:** **nfs** ou outro driver remoto pode ser necessário;
- **Necessidade de performance de leitura/escrita em memória:** **tmpfs** pode ser útil;
- **Ambiente em cloud com orquestrador:** drivers especializados como rexray com EBS ou plugins CSI no Kubernetes são boas opções.

Comece com o volume **local**. Ele atende bem na maioria dos cenários. Só parta para opções mais complexas se houver necessidade real de compartilhamento, persistência remota ou requisitos específicos de performance.

Comparação aprofundada: Docker volumes VS Bind Mount vstmpfs

Quando falamos de armazenamento em containers, o Docker oferece três mecanismos principais: **volumes**, **bindmounts** e **tmpfs**. Todos eles têm em comum

o fato de conectarem algo do host ao container, mas a forma como funcionam e os cenários onde se encaixam melhor são bem diferentes.

Volumes: a opção oficial, segura e portátil

Volumes são gerenciados pelo próprio Docker. Quando você cria um volume com `docker volume create`, ele é registrado nos metadados internos do Docker e fica salvo, por padrão, em `/var/lib/docker/volumes`. O que o container enxerga é o diretório `_data` desse volume.

Pontos fortes:

- Isolamento entre containers e o host;
- Gerenciamento fácil via comandos Docker;
- Backup e migração mais simples;
- Recomendado para ambientes de produção.

```
docker run -d --name app \
-v dados-app:/usr/src/app/data \
minha-imagem
```

Comando de Prompt 4 – Volumes no docker run
Fonte: elaborado pelo autor (2025)

Bind Mount: flexibilidade com riscos

Bindmounts apontam diretamente para um caminho específico no host. Isso significa que o container terá acesso a esse diretório exatamente como ele está, com todos os arquivos e permissões.

Pontos fortes:

- Ideal para desenvolvimento (acesso em tempo real a arquivos locais);
- Permite observar logs ou editar arquivos direto do host.

Desvantagens:

- Não há isolamento; o container pode afetar o sistema de arquivos do host;
- Portabilidade limitada, pois depende da estrutura de diretórios do host.

```
docker run -d --name app \
-v /home/user/projeto:/usr/src/app \
minha-imagem
```

Comando de Prompt 5 – Bind Mount

Fonte: elaborado pelo autor (2025)

tmpfs: efêmero, leve e volátil

Volumes **tmpfs** armazenam os dados apenas na RAM do host. Ao reiniciar ou parar o container, tudo é perdido. Isso é ótimo para informações temporárias ou cenários que precisam de desempenho sem gravar em disco.

Pontos fortes:

- Alta performance (acesso direto em memória);
- Nenhuma persistência (ideal para dados sensíveis ou efêmeros).

```
docker run -d --name app \
--tmpfs /tmp/dados \
minha-imagem
```

Comando de Prompt 6 – tmpfs
Fonte: elaborado pelo autor (2025)

Diferenças entre tipos de redes Docker

Quando falamos de redes Docker, estamos falando de como containers se comunicam entre si e com o mundo exterior. O Docker oferece diferentes "tipos" de rede para atender cenários variados de isolamento, performance e interoperabilidade. Entender esses tipos é essencial para montar ambientes bem conectados, seguros e eficientes.

Bridge: o padrão que funciona para muita coisa

Quando você não especifica uma rede, o Docker automaticamente coloca seu container na rede **bridge** padrão. Essa rede cria um isolamento de rede dentro do host e fornece resolução de nomes entre containers.

Vantagens:

- Permite comunicação entre containers no mesmo host;
- Cada container recebe um IP isolado;
- Tem DNS interno: você pode usar o nome do container como hostname.

Quando usar:

- Desenvolvimento local;

- Ambientes simples em que todos os containers estão no mesmo host.

```
docker network create --driver bridge minha-bridge
```

Comando de Prompt 7 – Bridge
Fonte: elaborado pelo autor (2025)

Host: quando o container precisa da rede do host

No modo **host**, o container simplesmente compartilha o stack de rede do host. Nada de NAT, nada de IP isolado. Ele é o próprio host.

Vantagens:

- Mais performance (sem NAT);
- Útil para containers que precisam de acesso direto à rede do host, como ferramentas de monitoramento.

Desvantagens:

- Zero isolamento de rede;
- Possível conflito de portas.

Quando usar:

- Containers que precisam expor portas conhecidas e fixas;
- Monitoramento, proxies locais, ou integração com apps nativos do host.

```
docker run --network host minha-imagem
```

Comando de Prompt 8 – Host
Fonte: elaborado pelo autor (2025)

None: para isolamento total

A rede **none** desativa completamente a conectividade do container. Nada de rede, sem ping, sem DNS, sem acesso ao mundo externo.

Vantagens:

- Isolamento total de rede.

Quando usar:

- Ambientes de teste;
- Aplicativos que não devem se comunicar com nada;

- Simulações de segurança e pentests.

```
docker run --network none minha-imagem
```

Comando de Prompt 9 – None
Fonte: elaborado pelo autor (2025)

Overlay e outras redes avançadas

Se você estiver usando Docker Swarm ou Kubernetes, vai ouvir falar de redes **overlay**. Elas permitem conexão entre containers em hosts diferentes. Mas isso já é papo pra outro momento, pois entra no mundo da orquestração.

Conclusão

- **Bridge** é o coringa do dia a dia: segura, isolada e com DNS interno;
- **Host** é para quando a performance ou integração com o host é prioridade;
- **None** é o modo silencioso absoluto.

E se você estiver orquestrando containers em cluster, vai querer aprender sobre **overlay**, **macvlan** e outros drivers mais avançados.

Escolher a rede certa evita dores de cabeça mais pra frente. Entender como elas funcionam te dá autonomia pra montar ambientes mais confiáveis, seguros e performáticos.

Criação e uso de redes personalizadas

Uma das grandes vantagens do Docker é poder simular ambientes com vários serviços isolados, mas que ainda assim precisam conversar entre si. E é exatamente aqui que entram as **redes personalizadas**. Com elas, você tem mais controle, isolamento e previsibilidade. Vamos entender o porquê.

Como funciona o DNS interno do Docker

Quando você cria uma rede personalizada com **docker network create**, o Docker automaticamente habilita um **serviço de DNS interno** para essa rede. Isso significa que cada container que estiver conectado a ela pode ser referenciado pelo seu nome (ou seja, o nome definido na flag **--name**).

```
docker network create minha-rede
```

```
docker run -d --name banco --network minha-rede postgres
```

```
docker run -d --name api --network minha-rede minha-api
```

Comando de Prompt 10 – DNS interno do Docker
Fonte: elaborado pelo autor (2025)

Nesse caso, a **minha-api** consegue se conectar ao banco de dados usando o hostname **banco**, sem precisar de IP fixo, sem um “jeitinho”. O Docker cuida da resolução de nomes.

Como containers se descobrem via hostname

Se os containers estão na **mesma rede personalizada**, o nome de um é visível para o outro. Isso simplifica muito a configuração de aplicações distribuídas.

Por exemplo, uma aplicação Node.js pode se conectar ao MongoDB apenas apontando a URI para **mongodb://mongo:27017**, assumindo que o container com o Mongo se chama **mongo** e ambos estão na mesma rede.

Como isolar e expor serviços de forma segura

Redes personalizadas são **isoladas por padrão**. Isso significa que apenas containers conectados à mesma rede conseguem se enxergar. Com isso, você pode:

- **Isolar bancos de dados**, evitando que sejam acessados por containers que não deveriam;
- **Separar ambientes** (ex: **dev** e **prod**);
- **Reduzir a superfície de ataque**, limitando o tráfego entre containers.

E caso precise expor um serviço (como um frontend ou API), basta mapear a porta com **-p** no **docker run**, ou configurar diretamente no Docker Compose.

```
docker run -d --name frontend --network minha-rede -p 80:3000 minha-app
```

Comando de Prompt 11 – Isolando e expondo serviços de forma segura
Fonte: elaborado pelo autor (2025)

Conectando e desconectando containers dinamicamente

Outro ponto importante é que você pode conectar e desconectar containers de redes a qualquer momento:

```
docker network connect minha-rede containerX  
docker network disconnect minha-rede containerY
```

Comando de Prompt 12 – Conectando e desconectando containers dinamicamente
Fonte: elaborado pelo autor (2025)

Isso permite criar arquiteturas dinâmicas, com controle fino sobre quem fala com quem.

Redes personalizadas não só organizam o tráfego entre seus containers, como também são uma ferramenta fundamental de segurança e isolamento. Entender como elas funcionam e como aproveitar o DNS interno do Docker muda completamente o jeito de estruturar suas aplicações multicontainer. E quando o projeto crescer? A mesma lógica de redes personalizadas será usada em ferramentas como Docker Compose e orquestradores como o Kubernetes.

Segurança com usuários não-root e permissões de arquivos

Criar containers seguros vai além de configurar uma imagem funcional. Um dos pilares mais importantes na segurança de ambientes Docker é evitar que containers rodem como **root**, além de configurar corretamente as permissões de arquivos dentro e fora do container.

Por que evitar o usuário root?

Quando um container é iniciado com o usuário **root**, ele tem permissões totais dentro daquele ambiente isolado. O problema é que, em caso de falha de segurança ou fuga de container (container breakout), esse usuário pode comprometer o host inteiro. A superfície de ataque aumenta consideravelmente.

Boas práticas:

- Nunca use **root** em containers que precisam interagir com dados sensíveis ou que vão rodar em produção;
- Prefira rodar com um usuário com permissões mínimas necessárias;
- Adicione esse usuário no momento da construção da imagem.

Como criar um usuário não-root na imagem Docker

Dentro do **Dockerfile**, você pode criar um usuário e definir que ele será o usuário padrão do container:

```
FROM node:18

# Cria o usuário
RUN useradd -m appuser

# Define permissões para a pasta da aplicação
WORKDIR /app
COPY . .
RUN chown -R appuser:appuser /app

# Troca para o usuário
USER appuser

CMD ["node", "index.js"]
```

Comando de Prompt 13 – Conectando e desconectando containers dinamicamente

Fonte: elaborado pelo autor (2025)

O comando **USER** muda o usuário efetivo usado durante a execução do container.

Permissões de arquivos e diretórios

Mesmo com um usuário não-root, é fundamental garantir que os arquivos e pastas tenham permissões adequadas.

- Use **chmod** e **chown** corretamente durante o build da imagem;
- Garanta que arquivos sensíveis (como tokens, **.env**, **.ssh**) estejam protegidos ou fora do contexto do container;
- Ao usar **volumes**, saiba que o usuário do host pode ser diferente do usuário do container — e isso pode gerar problemas de permissão;

Solução comum: mapear o mesmo UID/GID no host e no container, ou usar **--user \$(id -u):\$(id -g)** ao rodar o container.

Evitando escalonamento de privilégio

Mesmo com um usuário não-root, containers mal configurados podem ser explorados. Algumas proteções adicionais:

- Use `readOnlyRootFilesystem` (em ambientes orquestrados como Kubernetes);
- Não monte o socket do Docker (`/var/run/docker.sock`) dentro do container;
- Use `--cap-drop ALL` para remover todas as capabilities Linux e adicione apenas as estritamente necessárias;
- Habilite o AppArmor, SELinux ou seccomp no seu host.

Exemplo prático: rodando container com UID/GID definidos

```
docker run \  
  --rm \  
  --user $(id -u):$(id -g) \  
  -v $(pwd):/app \  
  minha-imagem
```

Comando de Prompt 14 – Rodando container com UID/GID definidos
Fonte: elaborado pelo autor (2025)

Esse comando garante que o usuário dentro do container terá os mesmos IDs do usuário do host, evitando problemas de permissão nos volumes montados.

Evitar o uso do `root` e gerenciar corretamente permissões dentro dos containers são passos cruciais para criar aplicações seguras e prontas para produção. Isso reduz a superfície de ataque e garante que falhas isoladas não se transformem em brechas críticas.

A segurança começa na imagem, passa pelo runtime e se estende até o host. Portanto, cuide de cada camada com atenção!

MERCADO, CASES E TENDÊNCIAS

Para compreender mais sobre o assunto, deixo aqui a sugestão de um ótimo livro:

“Docker: Up & Running” (3ª edição)

Autores: Sean P. Kane e Karl Matthias

Editora: O'Reilly Media

Ano: 2023

Esse livro é um prato cheio para quem já entende os fundamentos e quer dominar recursos como BuildKit, rootless containers, redes avançadas e integração com Kubernetes. Recomendo especialmente os capítulos sobre “Segurança”, “Redes” e “Práticas modernas de build” — eles complementam tudo o que vimos nessa aula.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você aprendeu a usar volumes e redes no Docker de forma mais estratégica, entendendo quando e como utilizar volumes nomeados, bindmounts e tmpfs. Também conheceu os diferentes tipos de redes (bridge, host, none), como criá-las, inspecioná-las e aplicá-las em cenários reais.

Além disso, praticou comandos essenciais para montar volumes, conectar containers por redes personalizadas e resolver problemas comuns no dia a dia com containers. Agora você tem mais controle sobre como seus containers armazenam dados e se comunicam entre si.

REFERÊNCIAS

DOCKER INC. **Networking overview.** 2024. Disponível em: <https://docs.docker.com/network/>. Acesso em: 20 ago. 2025.

DOCKER INC. **Storage overview.** 2024. Disponível em: <https://docs.docker.com/storage/>. Acesso em: 20 ago. 2025.

KANE, S.; MATTHIAS, K. **Docker: Up & Running.** Sebastopol: O'Reilly Media, 2023.

PALAVRAS-CHAVE

Contêineres. Virtualização. Orquestração.

EMAP



POSTECH